Node.js ile REST API Projesi

Backend geliştirme
bitirme ödevi teslim raporu

Proje adı: BookTrack — Kitap Yönetim REST API'si



Made with GAMMA

SENARYO VE SİSTEM DAVRANIŞI

1. Proje Senaryosu

BookTrack, kullanıcıların kitap kayıtlarını yönetebilmesini sağlayan bir REST API uygulamasıdır. Sistem, kitapların başlık, yazar, kategori ve yayın yılı bilgilerinin tutulmasını ve bu bilgiler üzerinde temel CRUD işlemlerinin gerçekleştirilmesini amaçlamaktadır.

Bir kullanıcı, kütüphanesindeki kitapları sisteme eklemek, mevcut kitapları listelemek, belirli bir kitabın detaylarını görüntülemek, kitap bilgilerini güncellemek veya artık ihtiyaç duymadığı bir kitabı silmek istemektedir.

Kullanıcı, bu işlemleri API endpoint'lerine HTTP istekleri göndererek gerçekleştirir. Sistem, gelen istekleri kontrol eder, gerekli doğrulamaları yapar ve işlem sonucunu uygun HTTP durum kodlarıyla birlikte JSON formatında döndürür.

Proje kapsamında veritabanı kullanılmadığından kitap kayıtları uygulama belleğinde bir dizi içerisinde tutulmaktadır.

Made with GAMMA

2. Sistem ne yapmalı

2.1. KİTAPLARI LİSTELEME

Senaryo: Kullanıcı, sistemde kayıtlı bütün kitapları görüntülemek ister.

Sistem davranışı:

- Kayıtlı kitapları JSON formatında döndürmelidir.
- İstek içerisinde yazar veya kategori filtresi bulunuyorsa sonuçları bu filtrelere göre daraltmalıdır.
- Geçersiz filtre değerlerinde uygun hata mesajı ve HTTP 400 yanıtı vermelidir.

2.2. KİTAP DETAYINI GÖRÜNTÜLEME

Senaryo: Kullanıcı, belirli bir kitabın bilgilerine ulaşmak ister.

Sistem davranışı:

- URL üzerinden gönderilen kitap ID'sini kontrol etmelidir.
- ID geçersizse HTTP 400 yanıtı vermelidir.
- Kitap bulunamazsa HTTP 404 yanıtı döndürmelidir.
- Kitap mevcutsa ilgili kitabın bilgilerini HTTP 200 yanıtıyla döndürmelidir.

2.3. YENİ KİTAP EKLEME

Senaryo: Kullanıcı, kütüphanesine yeni bir kitap kaydetmek ister.

Sistem davranışı:

- Kitap başlığı, yazar, kategori ve yayın yılı bilgilerini kontrol etmelidir.
- Zorunlu alanların boş bırakılmasını engellemelidir.
- Kategori ve yayın yılı değerlerinin geçerli olmasını sağlamalıdır.
- Yeni kitap için benzersiz bir ID oluşturmalıdır.
- Kitabı kayıt listesine eklemeli ve oluşturulan kitabı HTTP 201 yanıtıyla döndürmelidir.

2.4. KİTAP BİLGİLERİNİ GÜNCELLEME

Senaryo: Kullanıcı, kayıtlı bir kitabın başlığını, yazarını, kategorisini veya yayın yılını değiştirmek ister.

Sistem davranışı:

- Kitap ID'sinin geçerliliğini kontrol etmelidir.
- İlgili kitabın sistemde bulunup bulunmadığını doğrulamalıdır.
- Yalnızca istekte gönderilen alanları güncellemelidir.
- Gönderilen değerlerin geçerliliğini kontrol etmelidir.
- Güncelleme başarılı olduğunda güncel kitap bilgilerini HTTP 200 yanıtıyla döndürmelidir.

Made with GAMMA

2.5. KITAP SILME

Senaryo: Kullanıcı, artık kütüphanesinde tutmak istemediği bir kitabı silmek ister.

Sistem davranışı:

- Kitap ID'sinin geçerliliğini kontrol etmelidir.
- İlgili kitabın mevcut olup olmadığını doğrulamalıdır.
- Kitabı kayıt listesinden kaldırmalıdır.
- Silme işlemi başarılı olduğunda HTTP 200 veya uygulamanın tasarımına göre HTTP 204 yanıtı vermelidir.
- Kitap bulunamazsa HTTP 404 yanıtı döndürmelidir.

2.6. HATA YÖNETIMI VE İSTEK GÜNLÜĞÜ

Senaryo: Kullanıcı geçersiz veri gönderir veya istek işlenirken beklenmeyen bir hata oluşur.

Sistem davranışı:

- Hataları merkezi error handler middleware aracılığıyla yönetmelidir.
- Uygun HTTP durum kodunu ve anlaşılır bir hata mesajını döndürmelidir.
- Logger middleware aracılığıyla gelen HTTP istekleri hakkında kayıt tutmalıdır.
- Beklenmeyen hataların kontrolsüz biçimde uygulamanın çalışmasını etkilemesini önlemelidir.

BookTrack, kitap kayıtlarının HTTP istekleri aracılığıyla yönetilmesini sağlayan, JSON tabanlı bir REST API olarak çalışmalıdır. Sistem, geçerli istekleri başarıyla işlemeli; geçersiz istekleri uygun hata yanıtlarıyla karşılamalıdır. Katmanlı kod yapısı sayesinde yönlendirme, iş mantığı, veri yönetimi ve hata kontrolü birbirinden ayrılmalıdır.

Made with GAMMA

Middleware ve Hata Yönetimi

BookTrack uygulamasında HTTP isteklerinin kontrol edilmesi, istek bilgilerinin kaydedilmesi ve hataların merkezi olarak yönetilmesi amacıyla middleware yapısı kullanılmıştır.

Logger Middleware: Uygulamaya gelen HTTP isteklerinin izlenmesini sağlar. İsteklerin metodu, adresi ve diğer ilgili bilgileri takip ederek geliştirme ve hata ayıklama sürecini kolaylaştırır.

Error Handler Middleware: Uygulama sırasında oluşan hataları merkezi bir noktada karşılar ve uygun HTTP durum kodlarıyla JSON formatında hata yanıtları oluşturur. Böylece hata yanıtlarının daha tutarlı bir yapıda sunulması sağlanır.

Veri Doğrulama: Kitap ekleme ve güncelleme işlemlerinde gönderilen veriler kontrol edilir. Geçersiz ID, boş alan veya geçersiz kategori gibi durumlarda uygun hata yanıtları döndürülür.

Bu yapı sayesinde uygulamanın hata yönetimi düzenlenmiş, isteklerin izlenebilirliği artırılmış ve API yanıtlarının tutarlılığı desteklenmiştir.

Made with GAMMA

BookTrack API Endpoint’leri

HTTP Metodu	Endpoint	İşlev
GET	/api/books	Tüm kitapları listeler.
GET	/api/books?author=George%20Orwell	Belirli bir yazara ait kitapları listeler.
GET	/api/books?category=Roman	Belirli bir kategorideki kitapları listeler.
GET	/api/books/:bookId	ID üzerinden kitap detayını getirir.
POST	/api/books	Yeni kitap kaydı oluşturur.
PATCH	/api/books/:bookId	Mevcut kitabın gönderilen alanlarını günceller.
DELETE	/api/books/:bookId	Belirtilen kitabı siler.

Not: Yazar ve kategori filtreleri aynı istekte birlikte de kullanılabilir. Tüm isteklerde JSON formatı kullanılır.

Made with GAMMA

2. Postman İstekleri

Postman'de her istek için yeni bir request oluştur. Aşağıdaki isteklerde başlangıç verilerinde bulunan 1 ve 2 ID'lerini kullandım.

1. Tüm Kitapları Listeleme

Method: GET
URL: http://localhost:3008/api/books
Beklenen yanıt — 200 OK:


```
[
 { "id": 1, "title": "Suç ve Ceza", "author": "Fyodor Dostoyevski", "category": "Roman", "publicationYear": 1866 },
 { "id": 2, "title": "1984", "author": "George Orwell", "category": "Distopya", "publicationYear": 1949 }
]
```


2. Yazara Göre Kitap Listeleme

Method: GET
URL: http://localhost:3008/api/books?author=George%20Orwell
Beklenen yanıt — 200 OK:


```
[
 { "id": 2, "title": "1984", "author": "George Orwell", "category": "Distopya", "publicationYear": 1949 }
]
```


Made with GAMMA

Postman — Filtreleme ve kitap detayı

3. Kategoriye Göre Kitap Listeleme

Method: GET

URL: http://localhost:3008/api/books?category=Roman

Beklenen yanıt — 200 OK:


```
[
  { "id": 1, "title": "Suç ve Ceza", "author": "Fyodor Dostoyevski", "category": "Roman", "publicationYear": 1866 }
]
```


4. Kitap Detayını Getirme

Method: GET

URL: http://localhost:3008/api/books/1

Beklenen yanıt — 200 OK:


```
{ "id": 1, "title": "Suç ve Ceza", "author": "Fyodor Dostoyevski", "category": "Roman", "publicationYear": 1866 }
```


Made with GAMMA

5. Yeni Kitap Ekleme

Method: POST
URL: http://localhost:3008/api/books
Postman → Body → raw → JSON:

```
{ "title": "Kürk Mantolu Madonna", "author": "Sabahattin Ali", "category": "Roman", "publicationYear": 1943 }
```

Beklenen yanıt: 201 Created. Dönen yanıtta yeni kitabın ID'si ve diğer alanları bulunmalıdır.

6. Kitap Bilgisi Güncelleme

Method: PATCH
URL: http://localhost:3008/api/books/1
Body → raw → JSON:

```
{ "title": "Suç ve Ceza - Güncel Baskı" }
```

Beklenen yanıt: 200 OK. Bu istek yalnızca kitap başlığını günceller; diğer alanlar değişmeden kalır.

7. Kitap Silme

Method: DELETE
URL: http://localhost:3008/api/books/2
Beklenen yanıt: Uygulamanın tasarımına göre 200 OK veya 204 No Content.
Silme işleminden sonra GET http://localhost:3008/api/books/2 ile doğrulama yapılır; beklenen yanıt 404 Not Found.

Made with GAMMA

Hata Yönetimi Testleri

Raporunda yalnızca başarılı istekleri değil, hatalı istekleri de göstermen faydalı olur.

Test	İstek	Beklenen durum
Geçersiz ID	GET /api/books/abc	400 Bad Request
Bulunmayan kitap	GET /api/books/9999	404 Not Found
Geçersiz kategori	GET /api/books?category=Geçersiz	400 Bad Request
Boş kitap başlığı	POST /api/books — title: ""	400 Bad Request
Güncellenecek alan yok	PATCH /api/books/1 — {}	400 Bad Request

Başarılı ve hatalı isteklerin birlikte test edilmesi, API'nin doğrulama ve hata yönetimi davranışını kanıtlar.

Made with GAMMA